Ellis Workflow

How work moves through Ellis: one case, from the moment something on the platform breaks to the moment a person acts, and how each capability earns its next rung of autonomy. What is built today and what is still planned are marked apart.

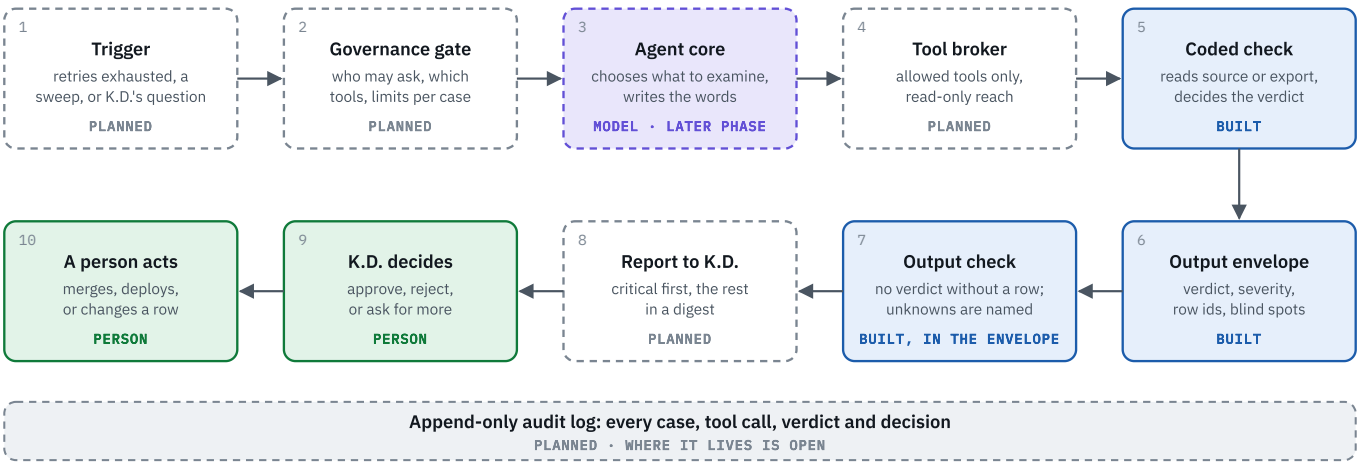
As of 10 Nov 2026 **Prepared by** Quincy Nkwain Ninying, Forward Deployed Engineer **Accountable** K.D. **Governance** A.M.
Platform read at a29dfb245

0 model calls at runtime in this phase; every verdict comes from a coded check	**40** / 48 capabilities verified by the platform from a commit that names the story	**16** / 16 sandbox rungs (R3 and R4) on the board, built and pushed	**17** shadow-mode rungs (R5 and up) waiting on K.D.'s acceptable misses and false alarms	**0** writes to the platform; every change is a person's

ONE CASE, END TO END

Ellis checks and reports; a person decides and acts

The target flow from K.D.'s architecture of 28 October. The coded checks and their output contract are built. The stages around them (entry points, governance gate, agent core, tool broker, report and audit log) are planned, and who builds them is an open question with A.M.



Built: coded, tested and pushed · Planned: in K.D.'s architecture, not built · Model step: comes in a later phase · A person's decision or action

THE SAME FLOW, READ AS STEPS

What happens at each stage, and where it lives

1. **Something starts a case** PLANNED
   A platform job that has run out of retries, a scheduled sweep, or a question from K.D. Ellis sends no email itself.

2. **The gate decides whether it may run** PLANNED
   Who may ask, which tools a case may use, and limits per case on tool calls, time and cost, proposed by K.D. and not yet confirmed.

3. **The agent core plans the look** LATER PHASE
   A model chooses what to examine and writes the words of the report. It never decides a verdict or a severity, and it has no direct reach into the platform.

4. **The broker hands it allowed tools only** PLANNED
   Each tool is one coded check, reached only through the broker, and every one reads; none writes.

5. **A coded check decides** BUILT
   It reads the platform's source at a pinned commit, or an export a person supplies, and decides the verdict in a fixed order, below. One check per built capability, each with its tests, in `src/capabilities/`.

6. **The answer goes into one envelope** BUILT
   Every tool returns the same shape: verdict, case, severity, the row ids it rests on, and its blind spots, with the tool's own answer unchanged inside (`src/output/toolOutput.ts`).

7. **The envelope refuses an unsupported answer** BUILT
   A problem or an ok that cites no row never leaves the tool, and a partly unknown answer must name what it could not see. Severity comes only from K.D.'s table; until it has rules, every severity is unassigned and says why.

8. **K.D. gets the report** PLANNED
   What broke, the evidence, the diagnosis and its confidence, the severity, and one proposed action. Critical alerts first, the rest in a digest.

9. **K.D. decides** PERSON
   Approve, reject, or ask for more. Some capabilities are a person's decision at every autonomy level, forever.

10. **A person acts** PERSON
    Merging and deploying are default-denied to agents on the platform, so a fix ships through a reviewed deploy by a person.

INSIDE A CODED CHECK

Unknown is decided before clean

A check that cannot see is never allowed to read as healthy. Each check asks these questions in this order, and stops at the first that holds.

ORDER	WHAT THE CHECK FINDS	VERDICT	WHAT THE ANSWER MUST CARRY
1	The source or export could not be read	**unknown**	Why it could not read, never "no problem"
2	Read, but part of what it needs could not be seen	**partly_unknown**	Each blind spot, named
3	A rule failed on rows it read	**problem**	The row ids, and a severity from K.D.'s table or the reason there is none
4	Every rule held on rows it read	**ok**	The row ids it rests on

THE AUTONOMY LADDER

How a capability earns each rung

The platform's ladder, with what each rung means there, the proof Ellis brings, and where Ellis stands. Finishing a rung promotes nothing on its own: K.D. signs R0 to R2, A.M. signs R3 and up.

RUNG	PLATFORM MEANING	PROOF ELLIS BRINGS	ELLIS TODAY
R0	Define the workflow, requirements, risks and human decisions	Discovery documents and a story per capability	every capability starts here
R1	Prepare isolated development, fixtures and baseline tests	A baseline test that fails for the reported reason, run in a locked sandbox	bug fix dispatch
R2	Answer questions using approved knowledge and records	A named passing test and a measured figure, verified by the platform from a commit that names the story	**40 of 48** capabilities verified by the platform; at R2 they report and recommend, and change nothing
R3	Perform approved, user-initiated sandbox tool operations	A run a named person starts, on sandbox copies, with a control that shows the check can fail	**3** rungs: a schema comparison, timer proposals applied to a sandbox registry, a backup restored into an isolated target
R4	Execute and verify stateful, multi-step sandbox tasks	The task run end to end on a sandbox fixture, beside a control or contrast, every key rule broken on purpose and caught	**13** rungs, plus auto-repair and ticket resolution as first releases
R5	Evaluate in shadow mode against approved reference outcomes	Results compared with outcomes K.D. approves, within misses and false alarms K.D. sets	**17** rung stories waiting on those thresholds. The scoring harness is built: it counts catches, misses and false alarms against approved references, and judges nothing until K.D. sets the limits. Observations come from a blind review sheet a named person signs

RUNG	PLATFORM MEANING	PROOF ELLIS BRINGS	ELLIS TODAY
R6+	A separately approved, supervised production pilot (R6), then automating only authorized action classes within limits (R7), and on to R9	Set per capability; some never leave a person's hands	not started; each needs its own approval

HOW ONE RUNG IS BUILT

The same eight steps, every time

Each of the 16 sandbox rungs went through this loop before it was pushed. A step that finds a gap sends the work back, not forward.

Step 1

Brief in plain words

What the rung does and what would prove it, said before any code.

Step 2

Read the platform

A read-only clone at a pinned commit; every line cited is re-checked at that commit.

Step 3

Build the fixture and the mechanism

On sandbox copies only. Nothing runs on the platform.

Step 4

Add a control that must fail

The platform's own code, a one-way contrast, or a truncated backup. If nothing can fail, nothing is proven.

Step 5

Check each case on its own

So a check that always says yes cannot pass.

Step 6

Break it on purpose

Every key rule is removed in turn and a test must catch it; a gap gets a test, or is removed and written down.

Step 7

Gate before pushing

The full suite, the type check, and a scan for anything that must not be public.

Step 8

Commit under the board's story id

CI runs again. The platform verifies a story from the commit; for rung ids, once A.M.'s change to the plan lands.

The loop catches Ellis's own mistakes too. The timer proposal rung found that Ellis's earlier proposal code offered a second registry row for a name that already had one; it is fixed. And when the platform fixes something Ellis reported, as it did for finding 54 on 28 October, Ellis's answers are corrected to match, not left describing the old code.

Sources: K.D.'s architecture of 28 Oct 2026 (entry points, gate, agent core, broker, report); the platform's autonomy ladder at `a29dfb245` (`backend/src/services/agentDomain/autonomyLadder.ts:58-69`); Ellis's output contract (`src/output/toolOutput.ts`, decision record 006); the capability catalog (`docs/capabilities/README.md`); the platform board's rung stories.

Ellis is an agent built in phases. This phase is its evidence and judgement layer, with no model calls at runtime; model-backed reasoning comes later under the same evidence contract and autonomy ceilings.